

Лабораторна робота № 12 з курсу

"Архітектура обчислювальних систем та комп'ютерна схемотехніка"

Виконав:

Студент групи ПМі-12

Козій Дем'ян

Тема: Програмування співпроцесора з використанням команд обчислення трансцендентних функцій для даних дійсного типу

Мета роботи: Ознайомитися з принципами роботи математичного співпроцесора і використати його можливості для обчислення трансцендентних функцій та реалізації розгалужень

Варіант: 11

Хід виконання роботи:

1. У середовищі Visual Studio створено простий проект на мові C++.
2. Реалізовано програму, яка обчислює трансцендентний вираз згідно зі своїм варіантом: $y_i = (6c - d * \sqrt{12/d}) / (c + a_i - 1)$. Програма складається з двох модулів: головний модуль C++ (забезпечує ввід даних та вивід результату) та модуль безпосередніх обчислень за допомогою асемблерної вставки `__asm`.
3. В асемблерній вставці використано команди математичного співпроцесора для роботи з дійсними числами. Для завантаження операндів у стек співпроцесора використано `fld`, для арифметичних дій — `fadd`, `fsub`, `fmul`, `fdiv` та `fsqrt` (для обчислення квадратного кореня).
4. Для доступу до елементів масивів `a` та `res` використано регістри `esi` та `edi` відповідно, з інкрементацією на 4 байти в кожній ітерації циклу `loop`.
5. Для перевірки правильності роботи асемблерного коду реалізовано аналогічний розрахунок засобами мови C++ у функції `calculateCpp`.

Програмний код:

```
#include <iostream>
#include <cmath>
#include <string>

using namespace std;

const int n = 5;

const float c6 = 6.0f;
const float c12 = 12.0f;
const float c1 = 1.0f;

float c, d;
float a[n];
float res[n];

void calculateASM()
{
    __asm
    {
        lea esi, a
        lea edi, res
        mov ecx, n
        finit
        iteration :

        fld c6
        fmul c
        fld c12
        fdiv d
        fsqrt
        fmul d
        fsubp st(1), st(0)

        fld[esi]
        fadd c
        fsub c1

        fdivp st(1), st(0)
        fstp[edi]

        add esi, 4
        add edi, 4
        loop iteration
    }
}
```

```
void enteringData()
{
    cout << " Input c : ";
    cin >> c;
    cout << " Input d : ";
    cin >> d;

    if (c <= d) {
        cout << " Warning: Condition c > d is not met!" << endl;
    }

    cout << " Input a[i] : " << endl;
    for (int i = 0; i < n; ++i)
    {
        cout << " a[" << i + 1 << "] = ";
        cin >> a[i];
    }
}

void printRes(string sym)
{
    cout << " Calculate in " << sym << endl;
    for (int i = 0; i < n; ++i)
    {
        cout << " result: [" << i + 1 << "] = " << res[i] << endl;
    }
}

void calculateCpp()
{
    for (int i = 0; i < n; i++)
    {
        res[i] = (6 * c - d * sqrt(12 / d)) / (c + a[i] - 1);
    }
}

int main()
{
    enteringData();
    calculateASM();
    printRes("ASM");
    calculateCpp();
    printRes("C++");
    system("pause");
    return 0;
}
```

Результати виконання програми:

Для перевірки коректності роботи було проведено тестування обох підходів до обчислень (за допомогою асемблерної вставки та стандартних засобів C++).

```
Input c : 2
Input d : 1
Input a[i] :
a[1] = 1
a[2] = 2
a[3] = 3
a[4] = 4
a[5] = 5
Calculate in ASM
result: [1] = 4.26795
result: [2] = 2.8453
result: [3] = 2.13397
result: [4] = 1.70718
result: [5] = 1.42265
Calculate in C++
result: [1] = 4.26795
result: [2] = 2.8453
result: [3] = 2.13397
result: [4] = 1.70718
result: [5] = 1.42265
Press any key to continue . . . |
```

Висновок:

Під час виконання лабораторної роботи № 12 було засвоєно принципи роботи з математичним співпроцесором (FPU) на рівні мови асемблера в середовищі Visual Studio. Опановано використання стекоподібної архітектури регістрів співпроцесора та базових арифметичних команд для обчислення трансцендентних виразів з дійсними числами (fld, fmul, fsqrt, fsub, fadd, fdivp). Практично закріплено навички передачі даних між програмою на C++ та асемблерною вставкою, а також взаємодію з масивами за допомогою адресації через регістри-вказівники. Завдяки дублюванню розрахунків засобами C++ вдалося успішно верифікувати правильність роботи розробленого асемблерного алгоритму.